



UNIVERSIDADE FEDERAL DE SÃO CARLOS (UFSCar)
CCGT | DComp | Campus Sorocaba

PROCESSAMENTO DE LINGUAGEM NATURAL – 2026/1
Prof. Dr. Tiago A. Almeida

PROJETO FINAL DE CURSO

Prazo para entrega: 25/06/2026 – 23:55

Categorização Automática de Documentos Jurídicos

O sistema judiciário brasileiro lida diariamente com um volume imenso de processos, o que torna a automação da triagem de documentos essencial para otimizar fluxos de trabalho e a gestão de demandas em larga escala. A digitalização desses processos permite o acesso contínuo aos dados, possibilitando a identificação de padrões textuais associados a diferentes tipos de peças processuais e de movimentações legais.

Nos últimos anos, o uso de técnicas de Processamento de Linguagem Natural (PLN) e de Aprendizado de Máquina (AM) tem revolucionado a análise de textos jurídicos, permitindo que modelos computacionais aprendam automaticamente a reconhecer classes de documentos a partir de dados brutos extraídos (muitas vezes por meio de OCR) de arquivos PDF. Essa integração entre PLN e fluxos jurídicos abre novas perspectivas para aplicações de automação.

O projeto em questão utiliza a base de dados **VICTOR**, um conjunto de documentos digitais do Supremo Tribunal Federal (STF) do Brasil. Os dados contêm milhares de recursos judiciais, permitindo o estudo prático de modelos de categorização automática em um domínio com vocabulário técnico e específico.

Cada amostra do conjunto de dados contém o texto extraído de páginas de documentos. As amostras foram rotuladas conforme o tipo de documento a que pertencem: *Acórdão*, *Recurso Extraordinário (RE)*, *Agravo de Recurso Extraordinário (ARE)*, *Despacho* e *Sentença*. Isso possibilita o desenvolvimento de modelos de PLN para o reconhecimento automático da categoria dessas peças.

A proposta deste trabalho, conduzido em parceria com a empresa Visagio, é analisar e explorar esses dados textuais para desenvolver modelos preditivos capazes de identificar automaticamente o tipo de documento jurídico. O conjunto oferece uma oportunidade realista para enfrentar os desafios de ruído de extração (erros de OCR, carimbos, assinaturas) e de textos longos, servindo como base para aplicações práticas em *LegalTech*.

O problema

A identificação automática de tipos de documentos a partir de arquivos digitais é um desafio de grande relevância nos contextos corporativo e jurídico. Os documentos apre-

sentam grande diversidade estrutural e textual, bem como documentos não rotulados. Além disso, o vocabulário e o tamanho dos textos variam amplamente, o que exige o uso de técnicas avançadas de PLN para a criação de modelos generalizáveis e precisos.

Neste projeto, os dados disponibilizados devem ser utilizados para construir modelos capazes de categorizar automaticamente a página ou documento do STF em uma das cinco categorias disponíveis, com base em seu conteúdo textual. Após a realização de etapas de análise e pré-processamento textuais (como limpeza de ruído de OCR, tokenização, remoção de *stop words*, lematização e representação computacional), será possível avaliar quais abordagens lidam melhor com as nuances do “juridiquês” e comparar o desempenho de diferentes algoritmos.

O objetivo final é investigar como textos extraídos de processos reais podem ser utilizados para a categorização automática de peças legais, contribuindo para o desenvolvimento de sistemas inteligentes.

Implementação

A implementação deverá ser realizada exclusivamente em Python, podendo fazer uso de bibliotecas padrão de manipulação de dados, aprendizado de máquina e PLN disponíveis na distribuição Anaconda e no ecossistema atual, como Pandas, Numpy, scikit-learn, NLTK, spaCy e HuggingFace/Transformers. Para a implementação do projeto, recomenda-se utilizar os métodos e técnicas já disponibilizados em bibliotecas.

Dentre as etapas do projeto, as oito seguintes deverão ser **obrigatoriamente** realizadas (não limitadas a):

1. **Análise de dados;**
2. **Pré-processamento e limpeza;**
3. **Representação computacional** (vetorial esparsa e vetorial densa (*embeddings*));
4. **Tarefas básicas de PLN** adequadas ao contexto (*e.g.*, NER, PoS, AS, SRL, IE, etc.);
5. **Modelos de categorização clássicos** (Engenharia de atributos e treinamento de modelos com *e.g.*, Naïve Bayes, Regressão logística, Máquinas de vetores de suporte, etc.);
6. **Modelos profundos de categorização** (*e.g.*, Redes Convolucionais para texto, Redes Recorrentes como BiLSTM, etc.);
7. **Adaptação de Modelos de Linguagem** baseados em Transformers (*e.g.*, BERT-Timbau, Sabiá, *Small Language Models* via PEFT/QLoRA, etc.).
8. **Análise dos resultados** (emprego de diferentes métricas).

O projeto deverá ser realizado por times de três alunos. Cada time será responsável por tomar todas as decisões técnicas que conduzam ao melhor resultado, devendo fornecer *scripts* gerais que executem os métodos e realizem as análises reportadas.

Bases de dados e referências

As bases de dados (partições do conjunto VICTOR) são compostas de textos sem processamento, divididas em cinco classes:

- **Acórdão:** 0;
- **Agravo de Recurso Extraordinário (ARE):** 1;
- **Despacho:** 2;
- **Recurso Extraordinário (RE):** 3; e
- **Sentença:** 4.

O arquivo *train.csv* representa o corpus de treinamento, sendo composto por três colunas: **identificador do documento (Id)**, **texto do documento (Body)** e a **classe a que o documento pertence (Category)**. Já o arquivo *test.csv* é composto apenas pela coluna **identificador do documento (Id)** e corresponde aos dados que deverão passar pela inferência dos modelos desenvolvidos para serem julgados na competição. Por fim, o documento *sample_submission.csv* apresenta um exemplo de estrutura a ser enviada à competição. Todos esses documentos estão disponíveis na página da competição no [Kaggle](#).

Protocolo experimental, análise dos métodos e resultados

A maior parte da nota do projeto será atribuída à análise realizada nos seguintes itens:

- Análise exploratória para melhor entendimento dos dados (*e.g.*, distribuição de classes, tamanho dos textos, vocabulário mais frequente, etc.);
- Avaliação de diferentes técnicas de pré-processamento de texto (*e.g.*, limpeza, normalização, vetorização, etc.);
- Análise de técnicas de representação de texto (*e.g.*, impacto do uso de TF-IDF, *n*-gramas *vs* *word embeddings* para aceleração do treinamento e melhoria semântica, etc.);
- Resolução de tarefas básicas de PLN para análise, limpeza e geração de atributos (*e.g.*, NER, PoS, AS, SRL, IE, etc.);
- Treinamento e análise de modelos preditivos (clássicos e profundos);
- Análise de desempenho individual (com foco em métricas robustas ao desbalançamento, como a F1-Score) e comparação dos resultados por meio de modelos estatísticos.

Para que os experimentos sejam reproduzíveis, espera-se que todas as decisões tomadas sejam devidamente reportadas e justificadas no relatório.

Formatação

A implementação consiste em um único notebook Python (`.ipynb`), denominado `main.ipynb`. Parte da implementação poderá ser feita em scripts Python (`.py`), que serão importados pelo código do notebook, o que facilitará a organização do trabalho.

Todos os arquivos enviados devem estar devidamente comentados, com o(s) nome(s) e RA(s) do(s) aluno(s) especificados no cabeçalho. Adicionalmente, as etapas do trabalho devem ser explicadas em células de texto (em Markdown).

Organização sugerida: o notebook poderá ser dividido em quatro seções: (i) análise exploratória, (ii) pré-processamento, (iii) experimentos e (iv) análise dos resultados. Da mesma forma, os códigos poderão ser organizados em scripts Python, de acordo com cada seção do notebook em que são utilizados. O time é livre para organizar seu código da maneira que desejar, mas o uso da estruturação limpa é fortemente recomendado.

Competição

O projeto deverá ser conduzido com foco na competição privada, organizada em parceria com a Visagio, disponível na [página da competição](#).

Relatório

O relatório deverá ser escrito em $\text{\LaTeX}$, no padrão de um artigo científico, em português, usando formato IEEE (disponível no AVA), e recomenda-se que contenha as seguintes seções:

1. **Resumo;**
2. **Introdução:** breve descrição do problema de classificação jurídica;
3. **Trabalhos relacionados:** breve resumo de pesquisas na área de *Legal NLP* (como o próprio artigo do VICTOR ou BERT para leis), indicando técnicas e *insights*;
4. **Dados e pré-processamento:** análise da base de dados e pré-processamentos textuais realizados;
5. **Materiais e Métodos:** breve descrição dos métodos e técnicas empregados, com ênfase no fluxo adotado para a solução;
6. **Protocolo experimental:** descrição da forma de avaliação, representação de texto, técnicas de PLN, métodos de classificação, ajustes de parâmetros e medidas de desempenho;
7. **Resultados:** descrição e análise dos resultados (apresentar tabelas e gráficos), incluindo os desempenhos obtidos no *Public Leaderboard*;
8. **Estratégia final:** detalhar as duas soluções finais escolhidas e enviadas na competição;
9. **Conclusões:** resumo de resultados obtidos e sugestões de extensão; e
10. **Referências:** lista de trabalhos consultados.

O relatório poderá conter, no máximo, 4 (quatro) páginas em coluna dupla (sem contabilizar as Referências). É importante que a solução adotada seja reprodutível, garantindo que todas as decisões tomadas tenham sido reportadas.

Composição da nota

A nota final do projeto será calculada por:

$$P = \frac{R}{10} \times (I + C)$$

sendo que:

- **R** corresponde à nota no relatório (0.0 a 10.0);
- **I** corresponde à nota na implementação (0.0 a 10.0);
- **C** corresponde a uma nota adicional dependendo da posição final na competição (Pos) no *Private Leader Board*. Ela será atribuída da seguinte forma:

$$C = \begin{cases} 1.00, & \text{se } Pos = 1 \\ 0.80, & \text{se } Pos = 2 \\ 0.60, & \text{se } Pos = 3 \\ 0.40, & \text{se } Pos = 4 \\ 0.20, & \text{se } Pos = 5 \\ 0.00, & \text{se } Pos > 5 \end{cases}$$

CUIDADOS

Leia atentamente os itens a seguir.

1. O projeto deverá ser enviado pelo AVA observando rigorosamente as seguintes regras:
 - Submeter um arquivo .zip chamado {RAs}-PROJ-PLN.zip, sendo que RAs corresponde aos RAs dos integrantes do time. Ex: 123456-654321-112233-PROJ-PLN.zip. Tal arquivo deverá conter uma pasta chamada {RAs}-PROJ-PLN, composta por duas subpastas: IMPLEMENTACAO (com o código-fonte completo e os arquivos necessários) e RELATORIO (com todos os arquivos $\text{\LaTeX}$ e pdf).
 - Não inclua as bases de dados no arquivo .zip. A sua implementação deverá ser capaz de reproduzir todos os passos a partir da base original no diretório configurado.
2. Apenas um membro do time precisará submeter o projeto no AVA;
3. Não utilize acentos nos nomes de arquivos;
4. Identificadores de variáveis: escolha nomes apropriados;

5. Documentação: inclua cabeçalho, comentários e indentação;
6. Erros de compilação/execução: nota zero no projeto;
7. Erros de compilação do relatório: nota zero no projeto;
8. Tentativa de fraude: qualquer tentativa de fraude implicará em nota zero na média para todos os envolvidos.